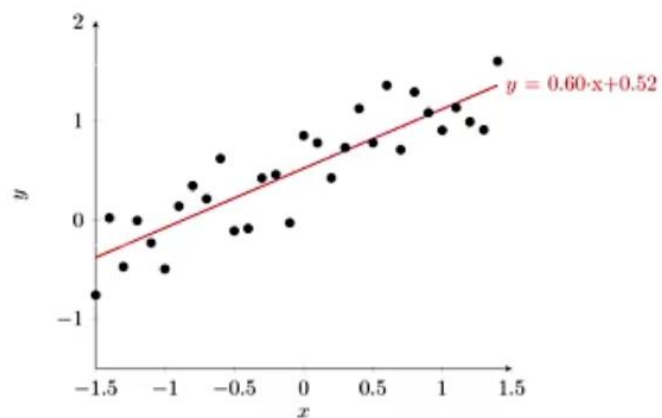


DS4023 (2025 Fall)
Machine Learning
Department of Computer Science
BNU-HKBU United International College



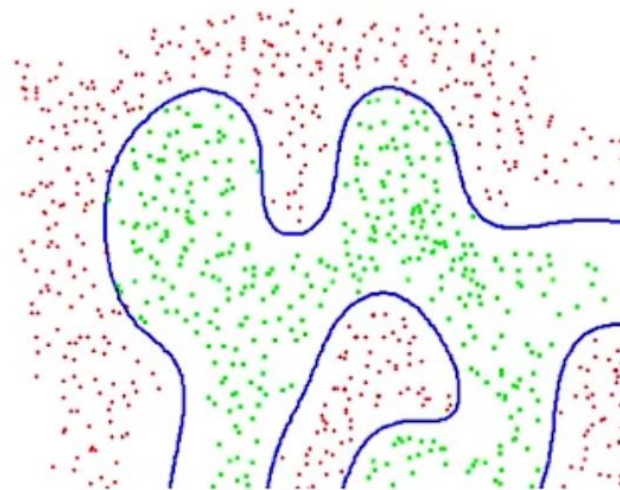
Chapter 3: Supervised Learning

Typical Supervised Learning



回归

Regression



分类

Classification



Chapter 3.1: Logistic Regression

Outline

- ▶ **Classification**
- ▶ Cost function and gradient
- ▶ Multi-class
- ▶ Regularization

Classification

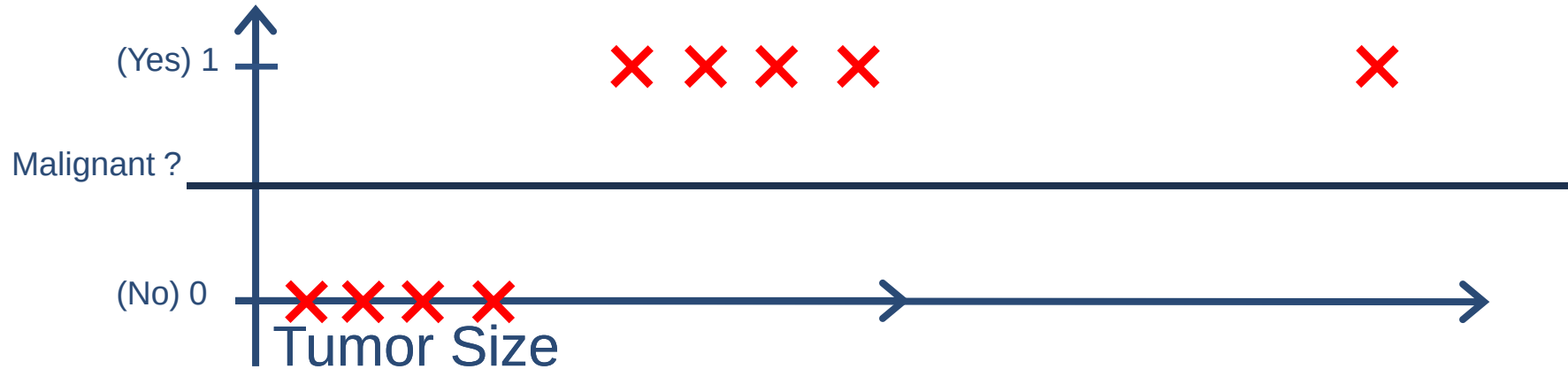
- ▶ Email: Spam / Not Spam?
- ▶ Online Transactions: Fraudulent (Yes / No)?
- ▶ Tumor: Malignant / Benign ?

$$y \in \{0,1\}$$

0: “Negative Class” (e.g., benign tumor)

1: “Positive Class” (e.g., malignant tumor)

Classification



Threshold classifier output $h_{\theta}(x)$ at 0.5:

If $h_{\theta}(x) \geq 0.5$, predict “ $y = 1$ ”

If $h_{\theta}(x) < 0.5$, predict “ $y = 0$ ”

Classification

Classification: $y = 0$ or 1

$h_{\theta}(x)$ can be > 1 or < 0

Logistic Regression: $0 \leq h_{\theta}(x) \leq 1$

Hypothesis Representation

Logistic Regression Model

Want $0 \leq h_{\theta}(x) \leq 1$

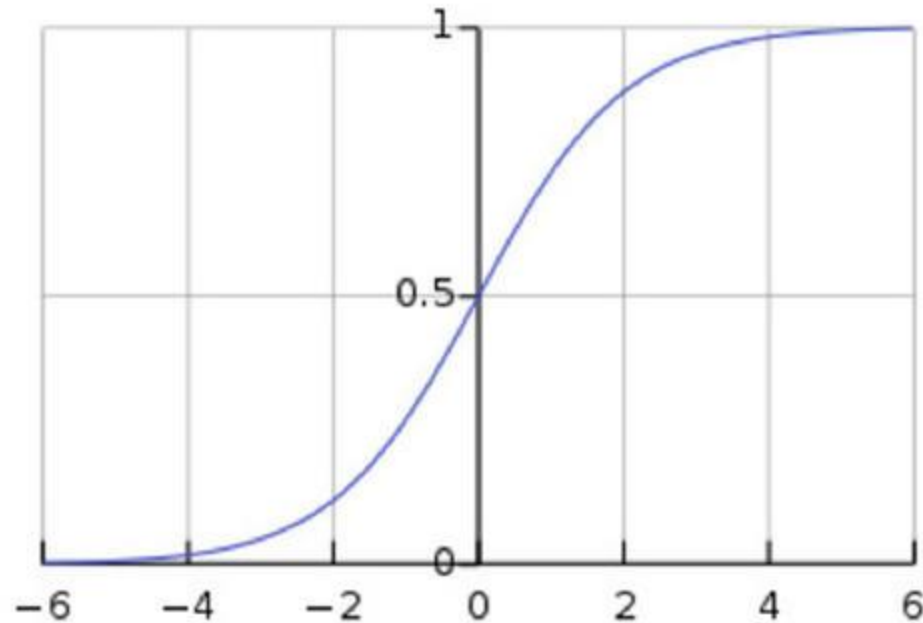
$$h_{\theta}(x) = \theta^T x$$



$$h_{\theta}(x) = g(\theta^T x)$$

Where, $g(z) = \frac{1}{1+e^{-z}}$

Sigmoid function
Logistic function



Interpretation of Hypothesis Output

$h_{\theta}(x)$ = estimated **probability** that $y = 1$ on input x

Example: If $x = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} 1 \\ tumorSize \end{bmatrix}$

$$h_{\theta}(x) = 0.7$$

Tell patient that 70% chance of tumor being malignant

$h_{\theta}(x) = P(y = 1|x; \theta)$ “probability that $y = 1$, given x , parameterized by θ ”

$$P(y = 0|x; \theta) + P(y = 1|x; \theta) = 1$$

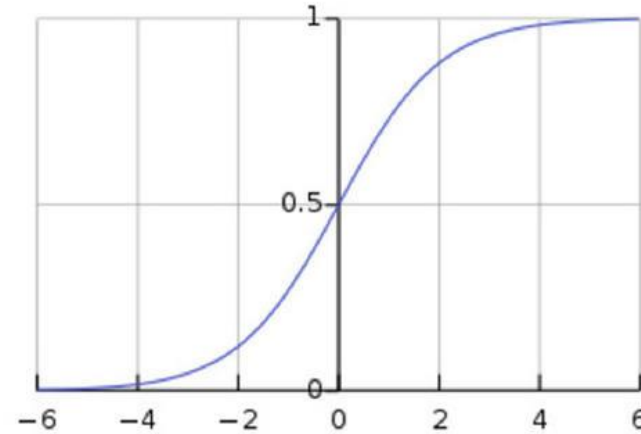
$$P(y = 0|x; \theta) = 1 - P(y = 1|x; \theta)$$

Decision boundary

Logistic regression

$$h_{\theta}(x) = g(\theta^T x)$$

$$g(z) = \frac{1}{1 + e^{-z}}$$

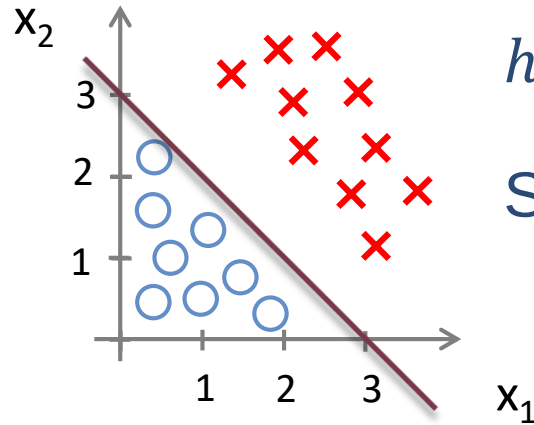


Suppose predict "y = 1" if $h_{\theta}(x) \geq 0.5$ ($\theta^T x \geq 0$)

predict "y = 0" if $h_{\theta}(x) < 0.5$ ($\theta^T x < 0$)

$\theta^T x = 0$: decision boundary

Decision boundary



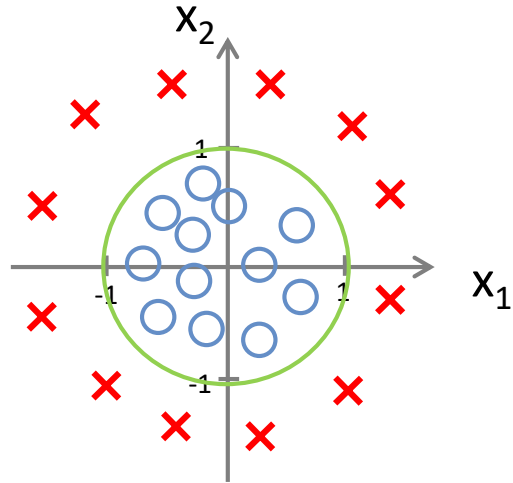
$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$

Suppose we have $\theta_0 = -3, \theta_1 = 1, \theta_2 = 1$

Predict “ $y = 1$ ” if $-3 + x_1 + x_2 \geq 0$

Predict “ $y = 0$ ” otherwise

Non-linear decision boundaries

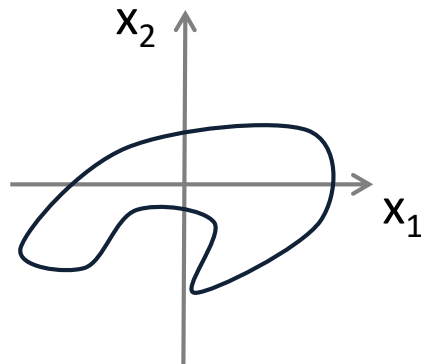


$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2)$$

- Choose $\theta_0 = -1, \theta_1 = 0, \theta_2 = 0, \theta_3 = 1, \theta_4 = 1$

Predict “ $y=1$ ” if $-1 + x_1^2 + x_2^2 \geq 0$

Predict “ $y=0$ ” otherwise



$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2)$$

Outline

- ▶ Classification
- ▶ **Cost function and gradient**
- ▶ Multi-class
- ▶ Regularization

Cost function

Training set: $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})\}$

m examples $x \in \begin{bmatrix} x_0 \\ x_1 \\ \dots \\ x_n \end{bmatrix}$ $x_0 = 1, y \in \{0,1\}$

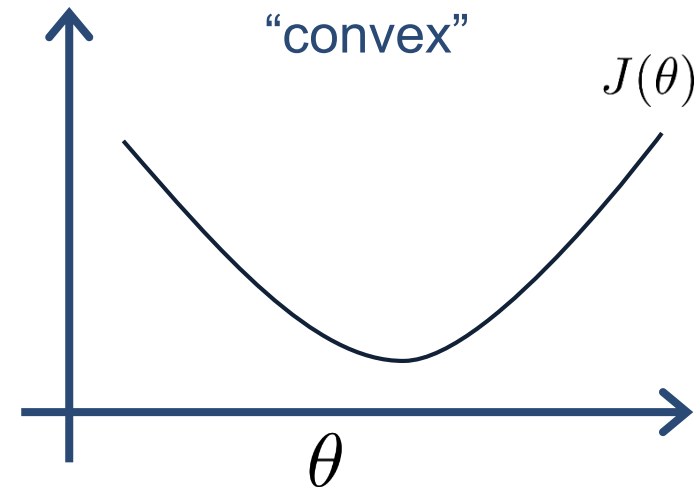
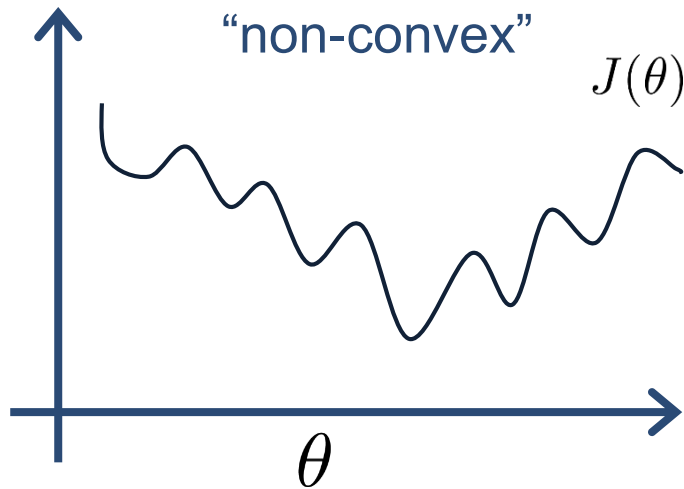
$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

How to choose parameters θ ?

Cost function

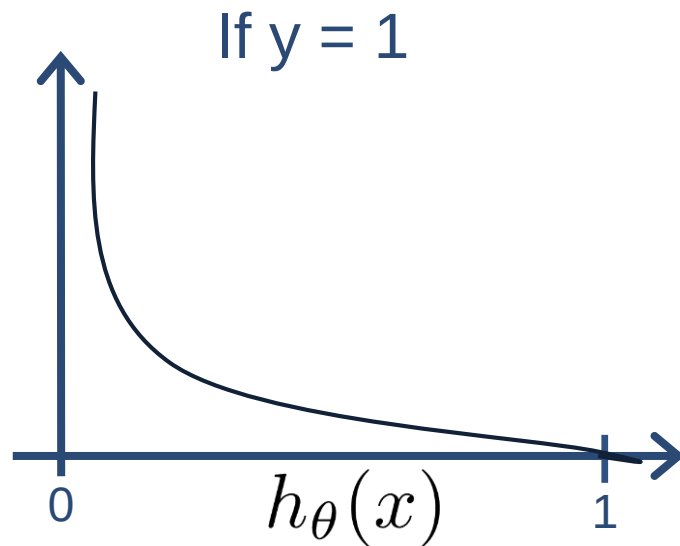
Linear regression: $J(\theta) = \frac{1}{m} \sum_{i=1}^m \frac{1}{2} (h_{\theta}(x^{(i)}) - y^{(i)})^2$

$$\text{Cost}(h_{\theta}(x^{(i)}), y^{(i)}) = \frac{1}{2} (h_{\theta}(x^{(i)}) - y^{(i)})^2$$



Logistic regression cost function

$$Cost(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$



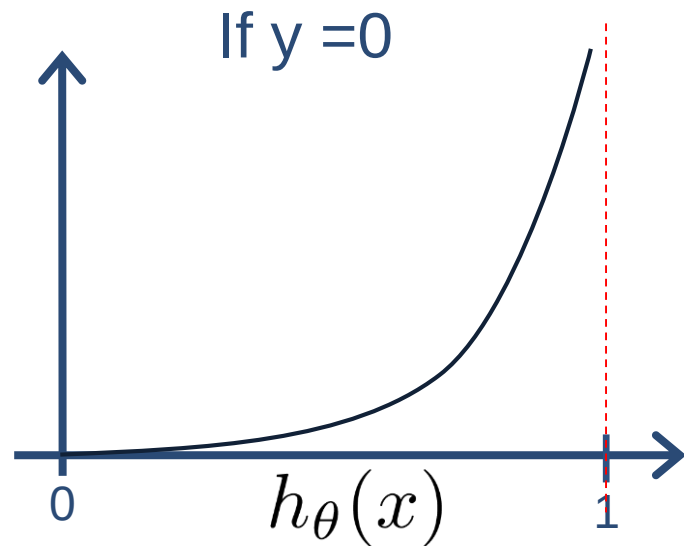
$Cost = 0$ if $y = 1, h_{\theta}(x) = 1$

But, as $h_{\theta}(x) \rightarrow 0, Cost \rightarrow \infty$

Captures intuition that if $h_{\theta}(x) = 0$ (predict $P(y = 1|x; \theta) = 0$), but $y = 1$, we will penalize learning algorithm by a very large cost.

Logistic regression cost function

$$Cost(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$



When $y = 0$,
Cost = 0, if $h_{\theta}(x) = 0$
Cost goes to infinite if $h_{\theta}(x) = 1$

Logistic regression cost function

Simplification:

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \text{Cost}(h_{\theta}(x^{(i)}), y^{(i)})$$

$$\text{Cost}(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$

Note: $y = 0$ or 1 always

$$\begin{aligned} J(\theta) &= \frac{1}{m} \sum_{i=1}^m \text{Cost}(h_{\theta}(x^{(i)}), y^{(i)}) \\ &= \frac{1}{m} \sum_{i=1}^m \left[-y^{(i)} \log(h_{\theta}(x^{(i)})) - (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right] \end{aligned}$$

Logistic regression cost function

$$\begin{aligned} J(\theta) &= \frac{1}{m} \sum_{i=1}^m \text{Cost}(h_{\theta}(x^{(i)}), y^{(i)}) \\ &= \frac{1}{m} \sum_{i=1}^m \left[-y^{(i)} \log(h_{\theta}(x^{(i)})) - (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right] \end{aligned}$$

To fit parameters θ :

$$\min_{\theta} J(\theta)$$

To make a prediction given new x :

$$\text{Output } h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

Gradient Descent

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \left[-y^{(i)} \log(h_{\theta}(x^{(i)})) - (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right]$$

Want $\min_{\theta} J(\theta)$:

Repeat {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

}

(simultaneously update all θ_j)

Notice the fact that for

$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = g(z)(1 - g(z))$$



Repeat {

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

}

(simultaneously update all θ_j)

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

Algorithm looks **identical** to linear regression!

Outline

- ▶ Classification
- ▶ Cost function and gradient
- ▶ **Multi-class**
- ▶ Regularization

Multiclass classification

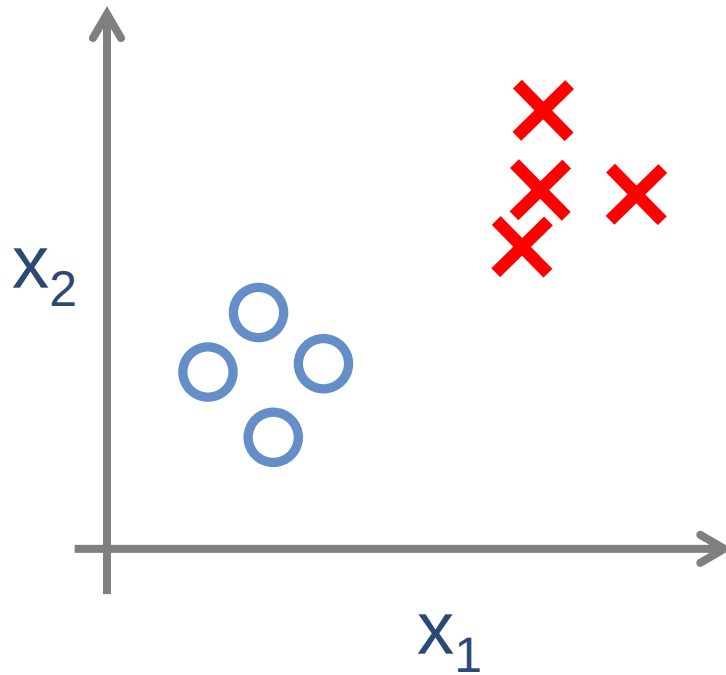
Email foldering/tagging: Work, Friends, Family, Hobby

Medical diagrams: Not ill, Cold, Flu, Coro-V

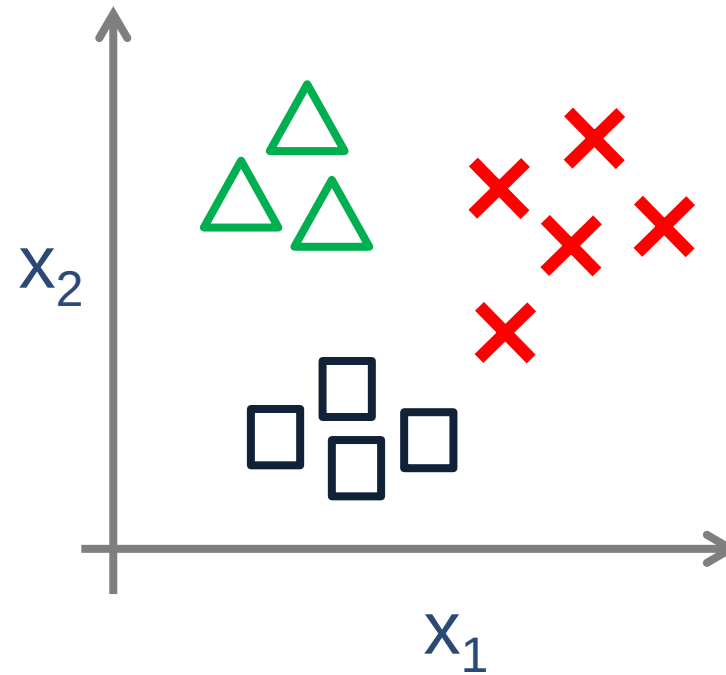
Weather: Sunny, Cloudy, Rain, Snow

Multiclass classification

Binary classification:

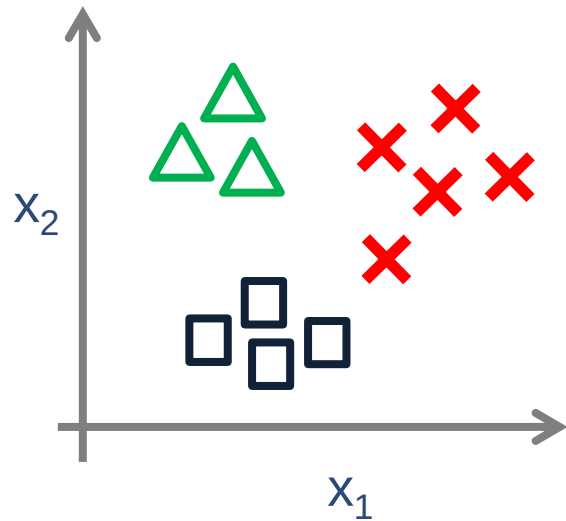


Multi-class classification:



Multiclass classification

One-vs-all (one-vs-rest):

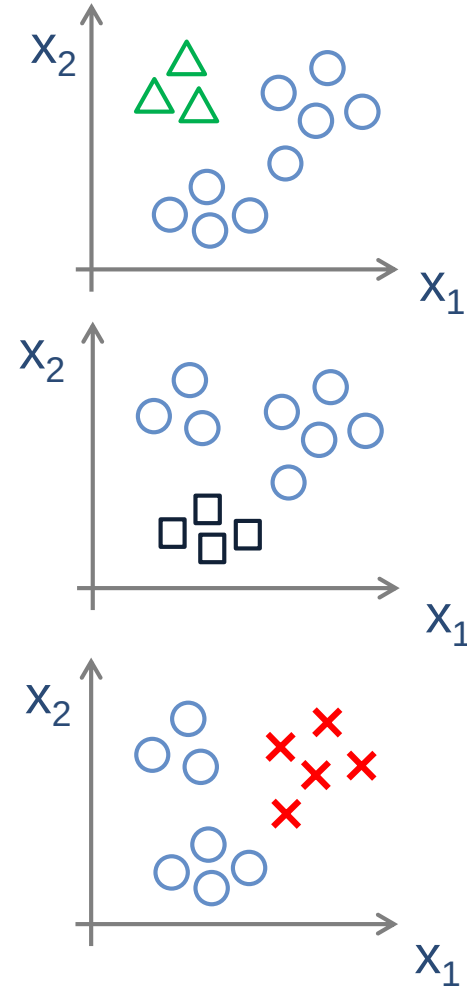


Class 1: 

Class 2: 

Class 3: 

$$h_{\theta}^{(i)}(x) = P(y = i|x; \theta), (i = 1, 2, 3)$$



Multiclass classification

Train a logistic regression classifier $h_{\theta}^{(i)}(x)$ for each class i to predict the probability that $y = i$.

On a new input x , to make a prediction, pick the class i that maximizes $h_{\theta}^{(i)}(x)$

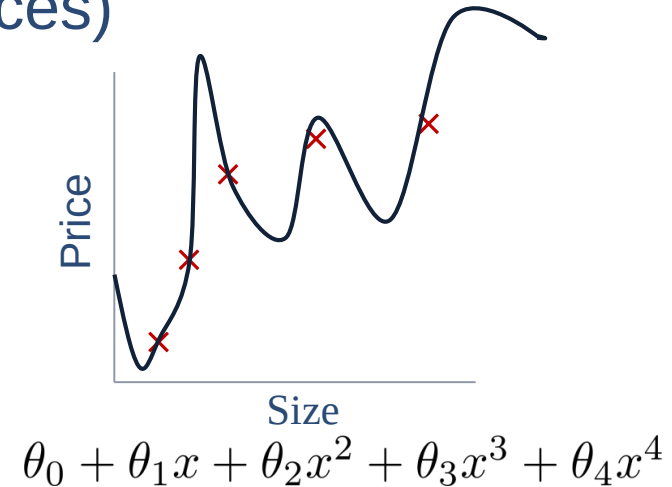
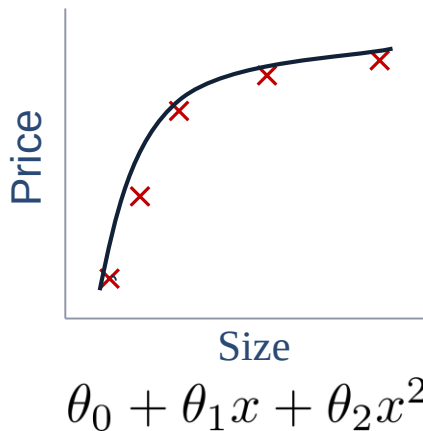
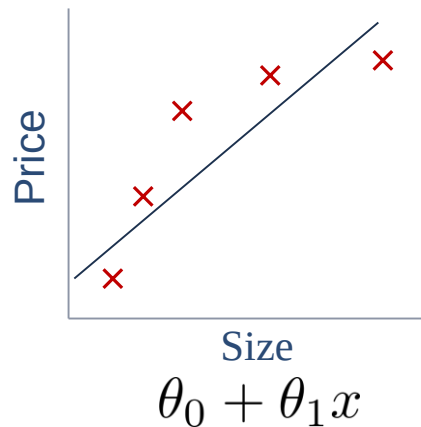
$$\max_i h_{\theta}^{(i)}(x)$$

Outline

- ▶ Classification
- ▶ Cost function and gradient
- ▶ Multi-class
- ▶ **Regularization**

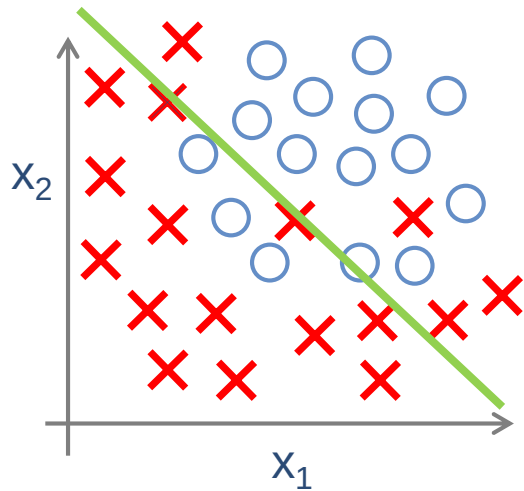
Regularization-The problem of overfitting

Example: Linear regression (housing prices)



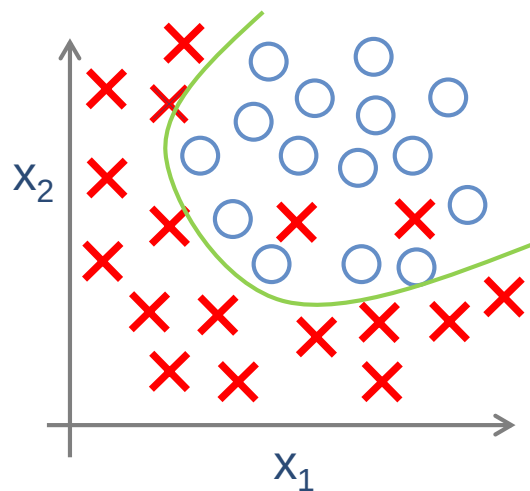
Overfitting: If we have too many features, the learned hypothesis may fit the training set very well ($J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \approx 0$), but fail to generalize to new examples (predict prices on new examples).

Example: Logistic regression

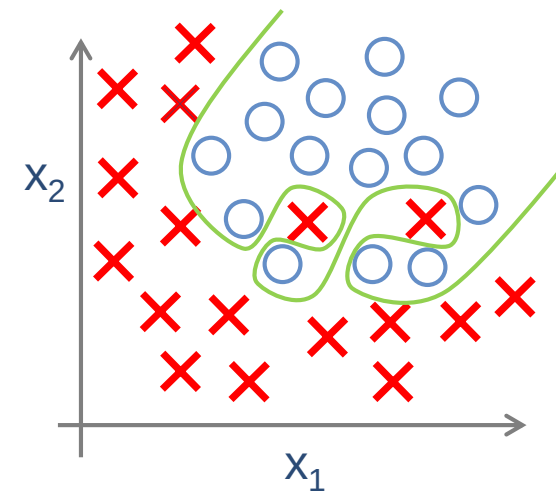


$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$

(g = sigmoid function)



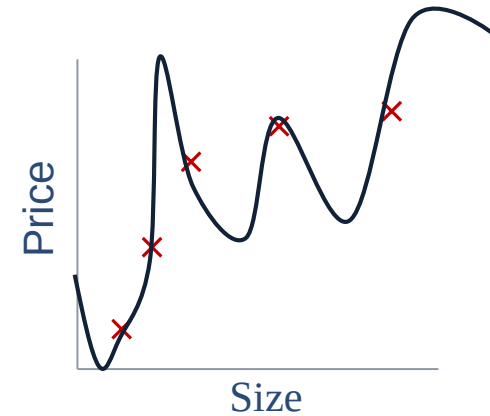
$$g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2)$$



$$g(\theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_1^2 x_2 + \theta_4 x_1^2 x_2^2 + \theta_5 x_1^2 x_2^3 + \theta_6 x_1^3 x_2 + \dots)$$

Addressing overfitting

x_1 = size of house
 x_2 = no. of bedrooms
 x_3 = no. of floors
 x_4 = age of house
 x_5 = average income in neighborhood
 x_6 = kitchen size
 $\vdots$
 x_{100}

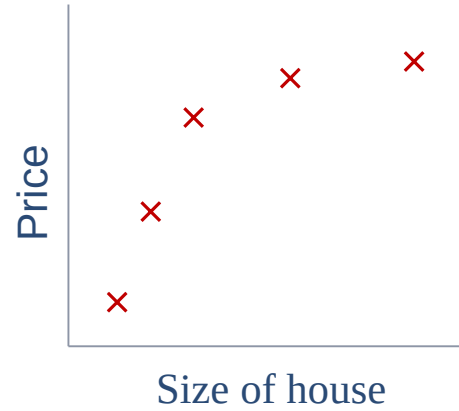


Addressing overfitting

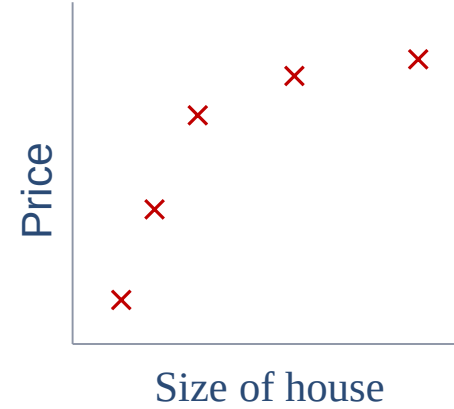
Options:

1. Reduce number of features.
 - Manually select which features to keep.
 - Model selection algorithm.
2. Regularization.
 - Keep all the features, but reduce magnitude/values of parameters.
 - Works well when we have a lot of features, each of which contributes a bit to predicting.

Intuition



$$\theta_0 + \theta_1 x + \theta_2 x^2$$



$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

$$\min_{\theta} \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + 1000\theta_3^2 + 1000\theta_4^2$$

Regularization

Small values for parameters $\theta_0, \theta_1, \dots, \theta_n$

- “Simpler” hypothesis
- Less prone to overfitting

Housing:

- Features: $x_0, x_1, \dots, x_{100}$
- Parameters: $\theta_0, \theta_1, \dots, \theta_{100}$

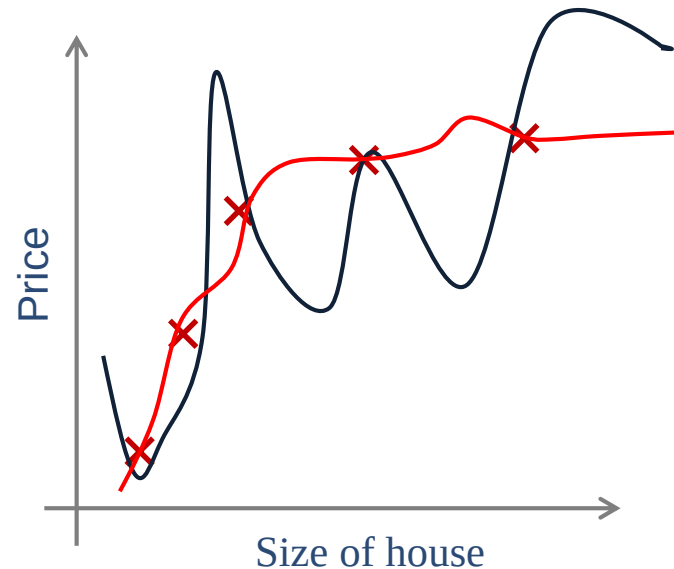
$$\min_{\theta} \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

Regularization

$$\min_{\theta} \frac{1}{2m} \left[\sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

$$\min_{\theta} J(\theta)$$

The two parts in the cost function make a balance between bias and **variance**, so that the model doesn't go underfitting or **overfitting** too much



Regularization

In regularized linear regression, we choose θ to minimize

$$\min_{\theta} \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

What if λ is set to an extremely large value (perhaps far too large for our problem, say $\lambda = 10^{10}$)?

- Algorithm works fine; setting λ to be very large can't hurt it
- Algorithm fails to eliminate overfitting.
- Algorithm results in underfitting. (Fails to fit even training data well).
- Gradient descent will fail to converge.

Regularization

In regularized linear regression, we choose θ to minimize

$$\min_{\theta} \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

What if λ is set to an extremely large value (perhaps far too large for our problem, say $\lambda = 10^{10}$)?



$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

All theta are penalized to almost 0

Regularization: linear regression

$$\min_{\theta} \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

$$\min_{\theta} J(\theta)$$

Gradient descent

Repeat {

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_0^{(i)}$$

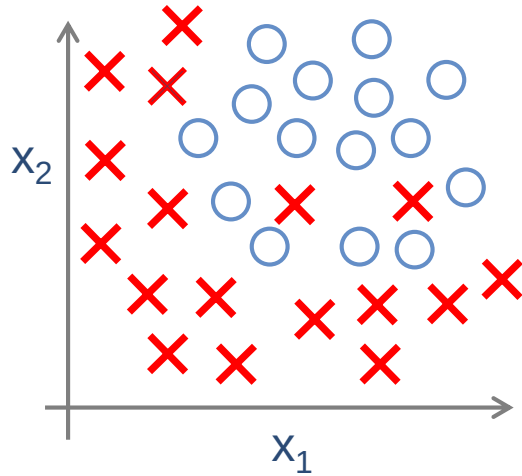
$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} - \alpha \frac{\lambda}{m} \theta_j$$

$$(j = 1, 2, \dots, n)$$

}

$$\theta_j := \theta_j \left(1 - \alpha \frac{\lambda}{m} \right) - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

Regularization: logistic regression



$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_1^2 x_2 + \theta_4 x_1^2 x_2^2 + \theta_5 x_1^2 x_2^3 + \dots)$$

Cost function:

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \left[-y^{(i)} \log(h_{\theta}(x^{(i)})) - (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

Regularization: logistic regression

Gradient descent

Repeat {

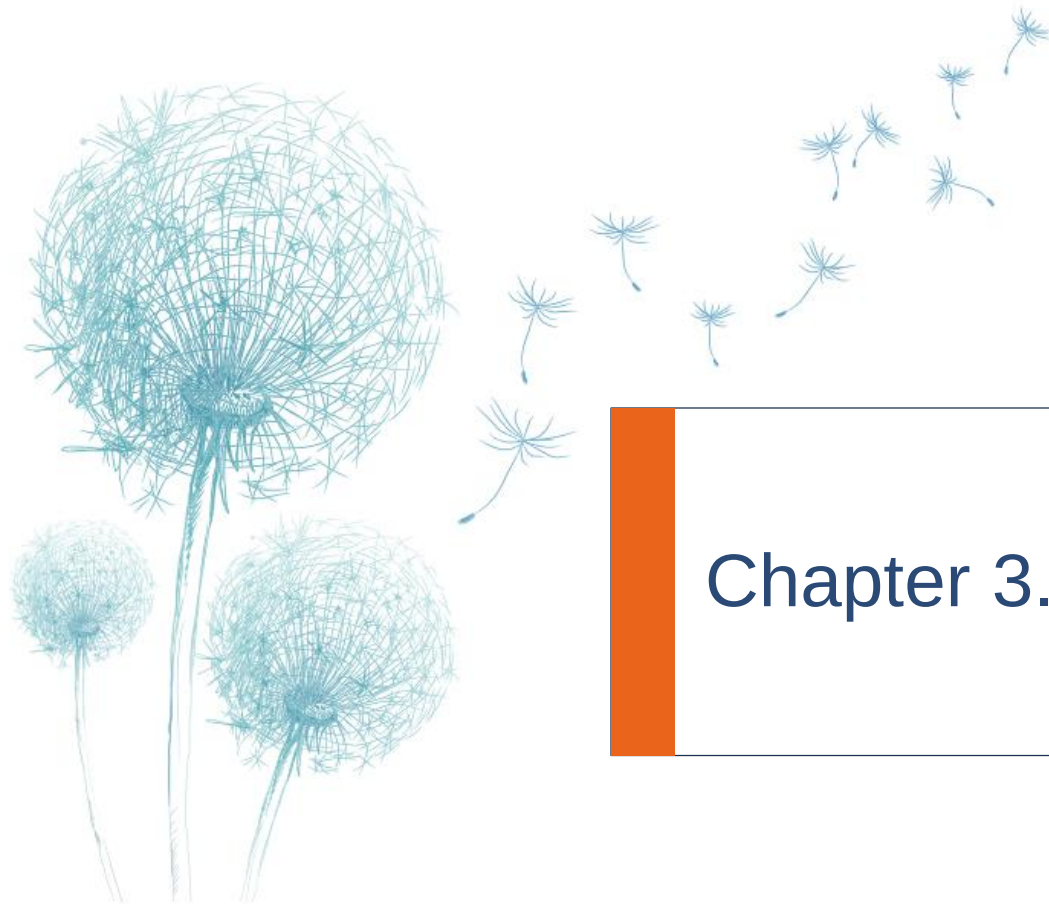
$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_0^{(i)}$$

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} - \alpha \frac{\lambda}{m} \theta_j$$

$$(j = 1, 2, \dots, n)$$

}

The above gradient formula looks exactly like what we have in linear regression, the only difference is the form of $h_{\theta}(x^{(i)})$.



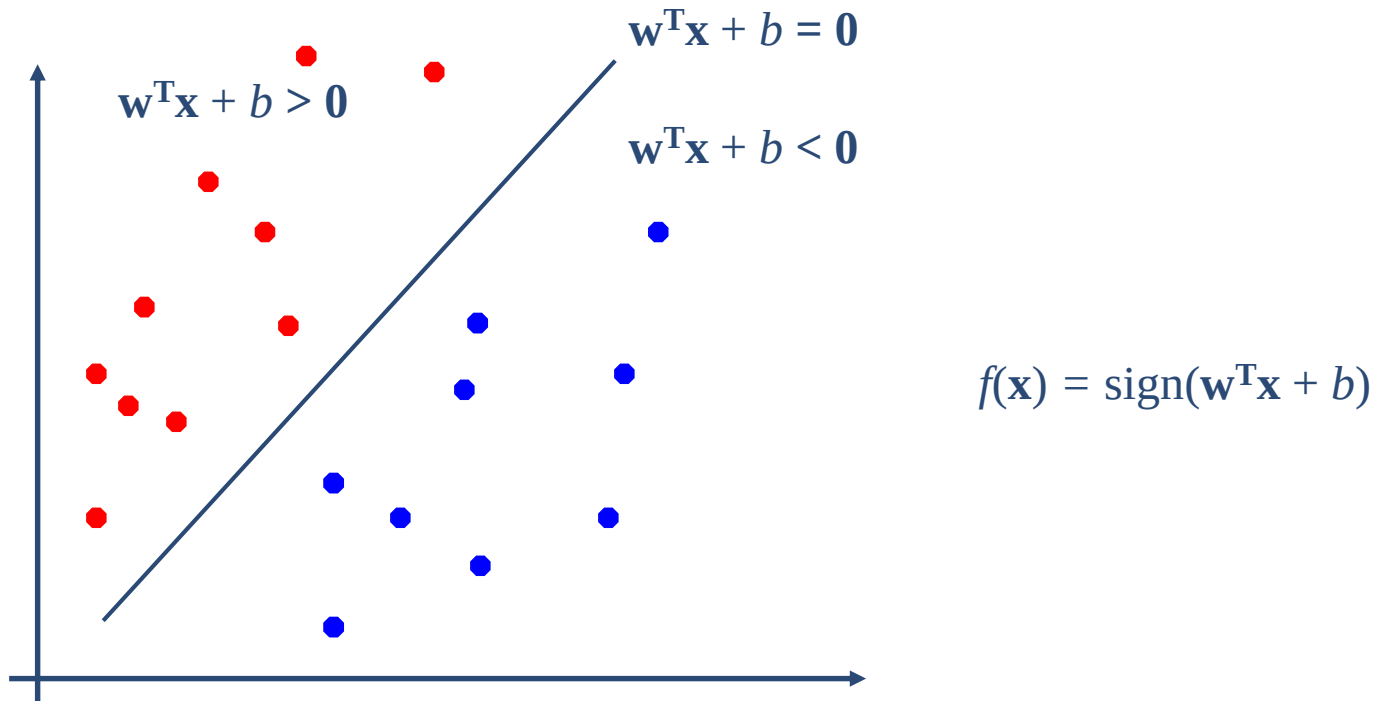
Chapter 3.2: Support Vector Machine

Outline

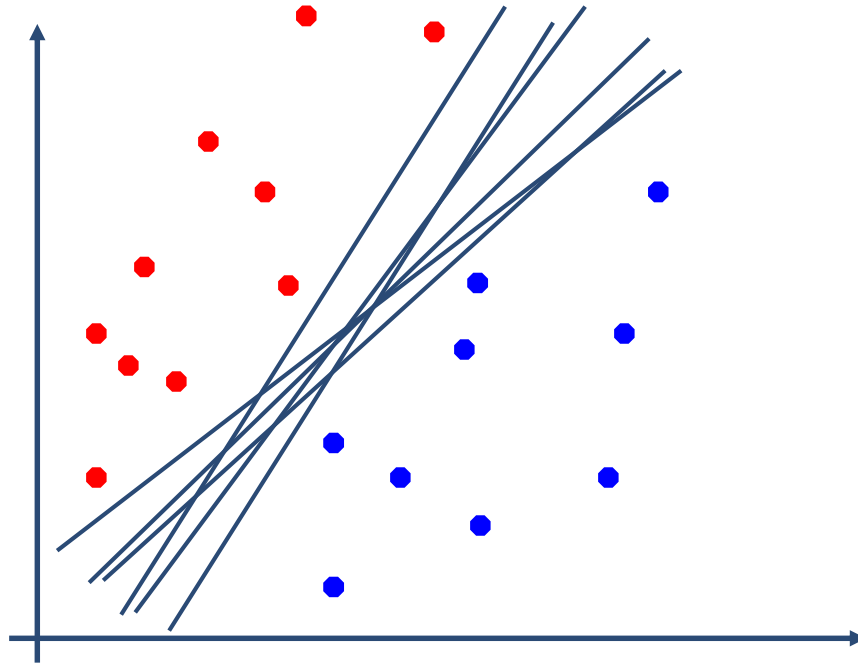
- ▶ Support vector machine model
- ▶ Constrained optimization
- ▶ Soft margin
- ▶ Kernel methods

Linear Separators

- Binary classification can be viewed as the task of separating classes in feature space:

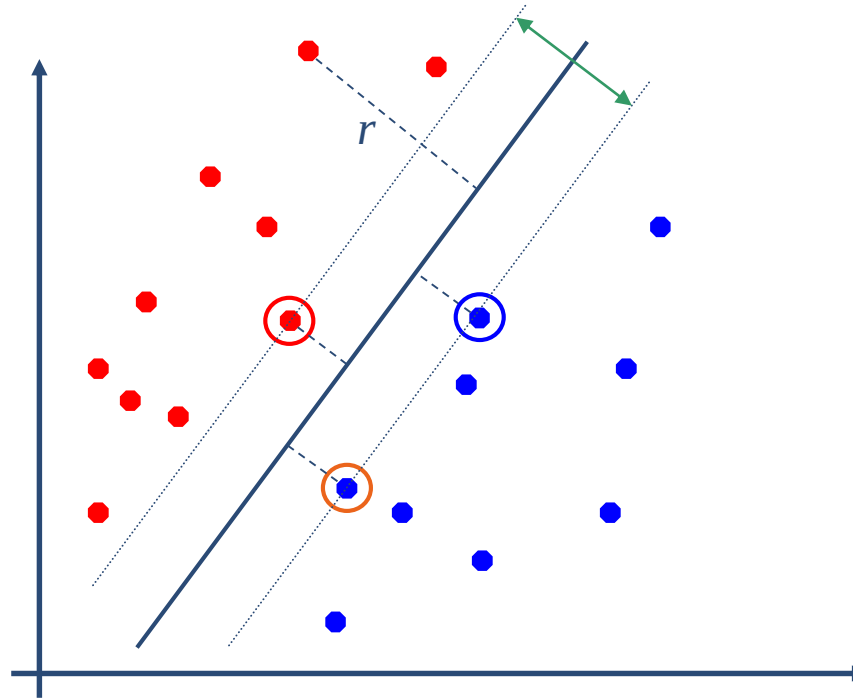


Which of the linear separators is optimal?



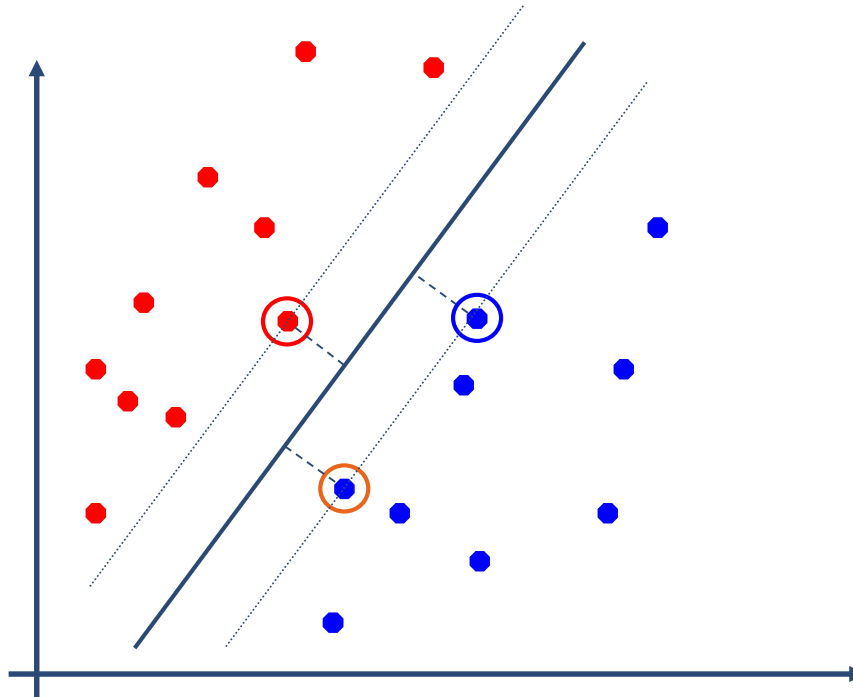
Classification Margin

- ▶ Distance from example $\mathbf{x}_i$ to the separator is $d = \frac{|\mathbf{w}^T \mathbf{x} + b|}{\|\mathbf{w}\|}$.
- ▶ Examples closest to the hyperplane are ***support vectors***.
- ▶ ***Margin*** γ of the separator is the distance between support vectors.



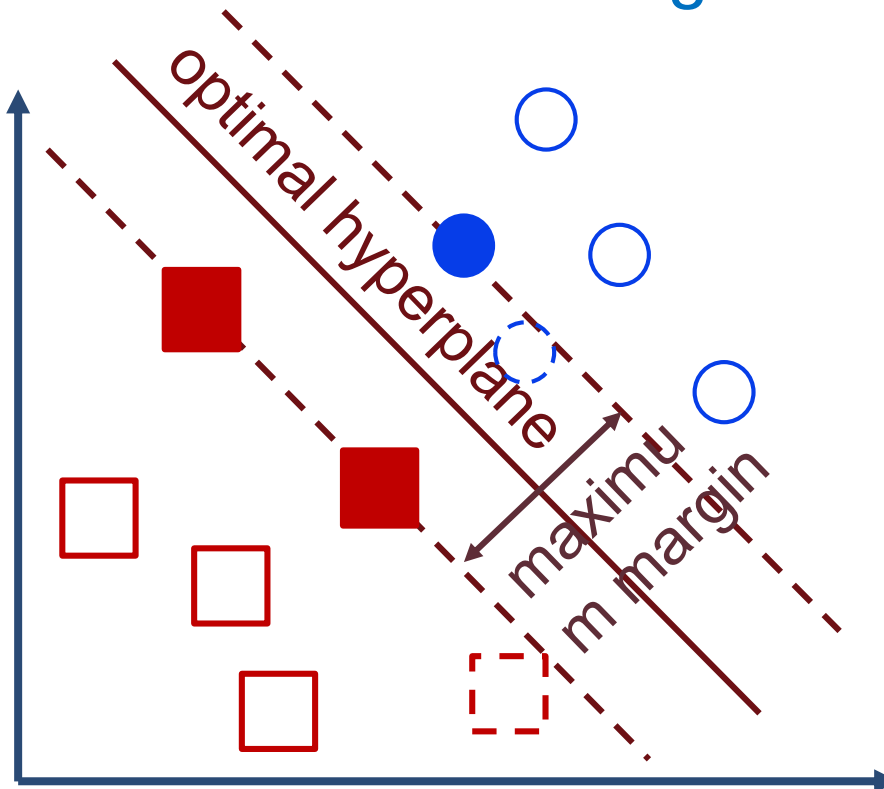
Maximum Margin Classification

- ▶ Maximizing the margin is good according to intuition and probably approximately correct (PAC) theory.
- ▶ Implies that only support vectors matter; other training examples are ignorable.



Goal of SVM

- ▶ The goal is to find the decision boundary with maximum margin
 - ▶ SVM also called **large margin classifier**
- ▶ This line (hyperplane) not only separates the two classes but also stays as far away from the **closest training instances** as possible



Linear SVM Mathematically

- ▶ Training set $\{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, ((\mathbf{x}_n, y_n))\}$
 - ▶ $\mathbf{x}_i \in \mathbb{R}^m$: input features
 - ▶ $y_i \in \{\pm 1\}$: input label
- ▶ Assume that the problem is linearly separable
 - ▶ there exists a linear surface to separate the two classes
 - ▶ 2-D: line; 3-D: plane; ... ; n-D: hyperplane ($\mathbf{w}^T \mathbf{x} + b = 0$)
 - ▶ $\mathbf{w} = (w_1, w_2, \dots, w_n)$: weight vector
 - ▶ b : scalar

Find a hyperplane that perfectly separates the two classes

Distance of a Point to Hyperplane

- ▶ A hyperplane is defined through $\mathbf{w}$ and b as a set of points such that $\mathcal{H} = \{\mathbf{x} | \mathbf{w}^T \mathbf{x} + b = 0\}$
 - ▶ $\mathbf{w} = (w_1, w_2, \dots, w_n)$: weight vector
 - ▶ b : scalar
- ▶ What is the distance (d) of a point $\mathbf{x}$ to hyperplane $\mathcal{H}$?
 - ▶ $d = \frac{|\mathbf{w}^T \mathbf{x} + b|}{\|\mathbf{w}\|}$, $\|\mathbf{w}\|$ is the norm/length of the vector
- ▶ Margin of $\mathcal{H}$ with respect to D (training set):
 - ▶ $\gamma(\mathbf{w}, b) = \min_{\mathbf{x} \in D} \frac{2|\mathbf{w}^T \mathbf{x} + b|}{\|\mathbf{w}\|}$
- ▶ The margin and hyperplane are scale invariant:
 - ▶ $\gamma(\beta \mathbf{w}, \beta b) = \gamma(\mathbf{w}, b), \forall \beta \neq 0$

Optimization Problem

- ▶ We can formulate our search for the maximum margin separating hyperplane as a constrained optimization problem
- ▶ The objective is to maximize the margin under the constraints that all data points must lie on the correct side of the hyperplane:

$$\begin{aligned} & \max_{\mathbf{w}, b} \gamma(\mathbf{w}, b) \\ & s. t. \quad y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 0, \quad \forall i \end{aligned}$$

Optimization Problem

- ▶ Because the hyperplane is **scale invariant**, we can fix the scale of $\mathbf{w}$, b anyway we want. Let's choose it such that

$$\min_{\mathbf{x} \in D} |\mathbf{w}^T \mathbf{x} + b| = 1$$

- ▶ Then our objective becomes:

$$\max_{\mathbf{w}, b} \gamma(\mathbf{w}, b) = \max_{\mathbf{w}, b} \left\{ \min_{\mathbf{x} \in D} \frac{2|\mathbf{w}^T \mathbf{x} + b|}{\|\mathbf{w}\|} \right\} = \max_{\mathbf{w}, b} \frac{2}{\|\mathbf{w}\|}$$

- ▶ The constraints are:

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 0, \quad \forall i$$

$$\min_{\mathbf{x} \in D} |\mathbf{w}^T \mathbf{x} + b| = 1$$

SVM model

- ▶ Consider the constraints:

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 0, \quad \forall i$$

$$\min_{\mathbf{x} \in D} |\mathbf{w}^T \mathbf{x} + b| = 1$$

$$\text{▶ } \mathbf{w}^T \mathbf{x}_i + b \begin{cases} \geq 0 \wedge \geq 1 & \text{if } y_i = 1 \\ \leq 0 \wedge \leq -1 & \text{if } y_i = -1 \end{cases}$$

- ▶ Therefore, the two constraints can be represented by:

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \forall i$$

- ▶ The optimization problem is equivalent to a simpler formulation (SVM model):

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

$$s. t. \quad y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \quad \forall i$$

- ▶ This can be solved with quadratic programming algorithm using Lagrange multipliers.

Lagrange Multiplier (Optional)

- ▶ Consider a problem of the following form:

$$\min_w f(w)$$

$$s.t. h_i(w) = 0, \quad i = 1, \dots, l$$

- ▶ Use the Lagrange multiplier method to solve the problem
- ▶ Define the Lagrangian to be $\mathcal{L}(w, \beta) = f(w) + \sum_{i=1}^l \beta_i h_i(w)$
 - ▶ β_i 's are called the Lagrange multipliers
 - ▶ Then solve the problem by set $\mathcal{L}$ partial derivative to 0, $\frac{\partial \mathcal{L}}{\partial w} = 0, \frac{\partial \mathcal{L}}{\partial \beta_i} = 0$

Lagrange Multiplier (Optional)

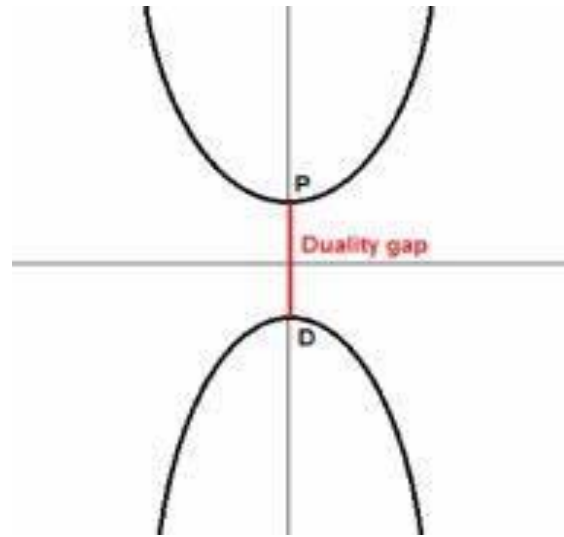
- ▶ Find the maximum and minimum values of $f(x, y) = 3x - 4y$ subject to the constraint $x^2 + 3y^2 - 129 = 0$.
- 1. Lagrangian: $\mathcal{L}(x, y, \beta) = 3x - 4y + \beta(x^2 + 3y^2 - 129)$
- 2. Find critical points
 - ▶ $\frac{\partial \mathcal{L}}{\partial x} = 3 + 2\beta x = 0$
 - ▶ $\frac{\partial \mathcal{L}}{\partial y} = -4 + 6\beta y = 0$
 - ▶ $\frac{\partial \mathcal{L}}{\partial \beta} = x^2 + 3y^2 - 129 = 0$
 - ▶ $\beta = \pm \frac{1}{6}$, two critical points $\{x = -9, y = 4\}, \{x = 9, y = -4\}$
- 3. Plug these into the function
 - ▶ $f(-9, 4) = -43$ (minimum)
 - ▶ $f(9, -4) = 43$ (maximum)

Duality (Optional)

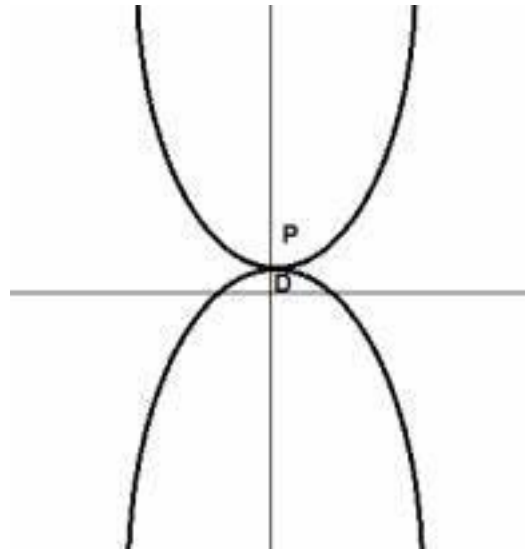
- ▶ In optimization theory, duality means that optimization problems may be viewed from either of two perspectives, the **primal problem** or the **dual problem**.
- ▶ The solution to the dual problem provides a **lower bound** to the solution of the primal (minimization) problem.
 - ▶ If you have a minimization problem, you can also see it as a maximization problem. And when you find the maximum of this problem, it will be a lower bound to the solution of the minimization problem.

Duality (Optional)

- ▶ Sometimes, solving the dual problem is simpler than solving the primal problem.
- ▶ Under certain convexity assumptions and suitable constraint qualifications, the primal and dual problems have **equal optimal objective values**.



Weak duality with duality gap



Strong duality without duality gap

Lagrange Duality (Optional)

- ▶ Consider a generalized constrained optimization problem which contains inequality as well as equality constraints.:

$$\begin{aligned} \min_w & f(w) \\ \text{s.t. } & g_i(w) \leq 0, \quad i=1, \dots, k \\ & h_i(w) = 0, \quad i=1, \dots, l \end{aligned}$$

- ▶ Also called **primal problem**. To solve it, we start by defining the generalized Lagrangian:

$$\mathcal{L}(w, \alpha, \beta) = f(w) + \sum_{i=1}^k \alpha_i g_i(w) + \sum_{i=1}^l \beta_i h_i(w)$$

- ▶ Here, the α_i 's and β_i 's are the Lagrange multipliers or dual variables
- ▶ **objective** is augmented with weighted sum of constraint functions

Lagrange Duality (Optional)

- ▶ Consider the quantity

$$\theta_{\mathcal{P}}(w) = \max_{\alpha, \beta: \alpha_i \geq 0} \mathcal{L}(w, \alpha, \beta)$$

- ▶ $\mathcal{P}$ stands for “**primal**”. Let w be given, if w violates any of the primal constraints then $\theta_{\mathcal{P}}(w) = \infty$; if the constraints are indeed satisfied for a particular value of w , then $\theta_{\mathcal{P}}(w) = f(w)$.

- ▶ Hence,

$$\theta_{\mathcal{P}}(w) = \begin{cases} f(w) & \text{if } w \text{ satisfied the primal constraints} \\ \infty & \text{otherwise} \end{cases}$$

Dual Problem (Optional)

- ▶ Hence, consider the minimization problem

$$\min_w \theta_{\mathcal{P}}(w) = \min_w \max_{\alpha, \beta: \alpha_i \geq 0} \mathcal{L}(w, \alpha, \beta)$$

- ▶ It is the same as our **original, primal** problem. Let optimal value of the primal problem be $p^* = \min_w \theta_{\mathcal{P}}(w)$

- ▶ Now, let's look at a slightly different problem:

$$\theta_D(\alpha, \beta) = \min_w \mathcal{L}(w, \alpha, \beta)$$

- ▶ New problem is optimizing (maximizing) w.r.t to α, β , which are minimized w.r.t. w .
- ▶ Then, we can have the **dual optimization problem**:

$$\max_{\alpha, \beta: \alpha_i \geq 0} \theta_D(\alpha, \beta) = \max_{\alpha, \beta: \alpha_i \geq 0} \min_w \mathcal{L}(w, \alpha, \beta)$$

- ▶ This is exactly the same as our primal problem shown above, except that the order of the "max" and the "min" are now exchanged.

Dual Problem (Optional)

- ▶ Let $d^* = \max_{\alpha, \beta: \alpha_i \geq 0} \min_w \mathcal{L}(w, \alpha, \beta) \leq \min_w \max_{\alpha, \beta: \alpha_i \geq 0} \mathcal{L}(w, \alpha, \beta) = p^*$
 - ▶ The solution to the dual problem provides a **lower bound** to the solution of the primal (minimization) problem
- ▶ Now, we can solve the **dual problem** in lieu of the primal problem
 - ▶ Easier for computation
 - ▶ Kernel functions
- ▶ Under the condition that f and g'_i 's are **convex functions** and h'_i 's are **affine**. And constraints of g_i are **strictly feasible**
 - ▶ There exists w^*, α^*, β^* , so that w^* is the solution to the primal problem and α^*, β^* are solution to the dual problem and $p^* = d^* = \mathcal{L}(w^*, \alpha^*, \beta^*)$
 - ▶ Moreover, w^*, α^*, β^* satisfy the KKT conditions

Karush-Kuhn-Tucker Conditions (Optional)

► KKT conditions are as follows:

$$\frac{\partial}{\partial w_j} \mathcal{L}(w^*, \alpha^*, \beta^*) = 0, \forall j \quad (1)$$

$$\frac{\partial}{\partial \beta_i} \mathcal{L}(w^*, \alpha^*, \beta^*) = 0, \forall i \quad (2)$$

$$\alpha_i^* g_i(w^*) = 0, \forall i \quad (3)$$

$$g_i(w^*) \leq 0, \forall i \quad (4)$$

$$\alpha_i^* \geq 0, \forall i \quad (5)$$

► Consider equation (3), it implies that if $\alpha_i^* > 0$, then $g_i(w^*) = 0$.
(Key for finding SVM support vectors!)

Optimal Margin Classifier (Optional)

- SVM optimization problem:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

$$s. t. \quad y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \quad i = 1, 2, \dots, m$$

- We can write the constraints as

$$g_i(w) = 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b) \leq 0$$

- When we construct the Lagrangian for our optimization problem we have:

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^m \alpha_i [1 - y_i(\mathbf{w}^T \mathbf{x}_i + b)]$$

- Let's find the dual form of the problem.
 - First **minimize** $\mathcal{L}(w, b, \alpha)$ with respect to w and b (for fixed α), to get $\theta_D(\alpha)$.

Dual Optimization Problem (Optional)

- We'll do by setting the derivatives of $\mathcal{L}$ with respect to w and b to zero:

$$\frac{\partial}{\partial w} \mathcal{L}(w, b, \alpha) = w - \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i = 0$$

$$\frac{\partial}{\partial b} \mathcal{L}(w, b, \alpha) = \sum_{i=1}^m -\alpha_i y_i = 0$$

- We have: $w = \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i$ and $\sum_{i=1}^m \alpha_i y_i = 0$, plug back into Lagrangian equation:

$$\begin{aligned} \mathcal{L}(w, b, \alpha) &= \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^m \alpha_i [1 - y_i (\mathbf{w}^T \mathbf{x}_i + b)] \\ &= \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j - b \sum_{i=1}^m \alpha_i y_i \\ &= \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \end{aligned}$$

Dual Optimization Problem (Optional)

- Recall that we got the equation by minimizing $\mathcal{L}$ with respect to w and b . Together with the constraint, we obtain the **dual optimization problem**:

$$\begin{aligned} \max_{\alpha} W(\alpha) &= \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \\ \text{s.t. } \alpha_i &\geq 0, \quad i = 1, 2, \dots, m \\ \sum_{i=1}^m \alpha_i y_i &= 0 \end{aligned}$$

- You can verify that the conditions required for $p^* = d^*$ and KKT conditions are satisfied. Therefore, we can solve the dual problem, which is a maximization with parameters α_i 's.

Dual Optimization Problem (Optional)

- ▶ Solve the problem with quadratic programming algorithm
- ▶ Or use SMO (sequential minimal optimization) algorithm to efficiently solve the dual problem.
- ▶ SMO simply does the following:

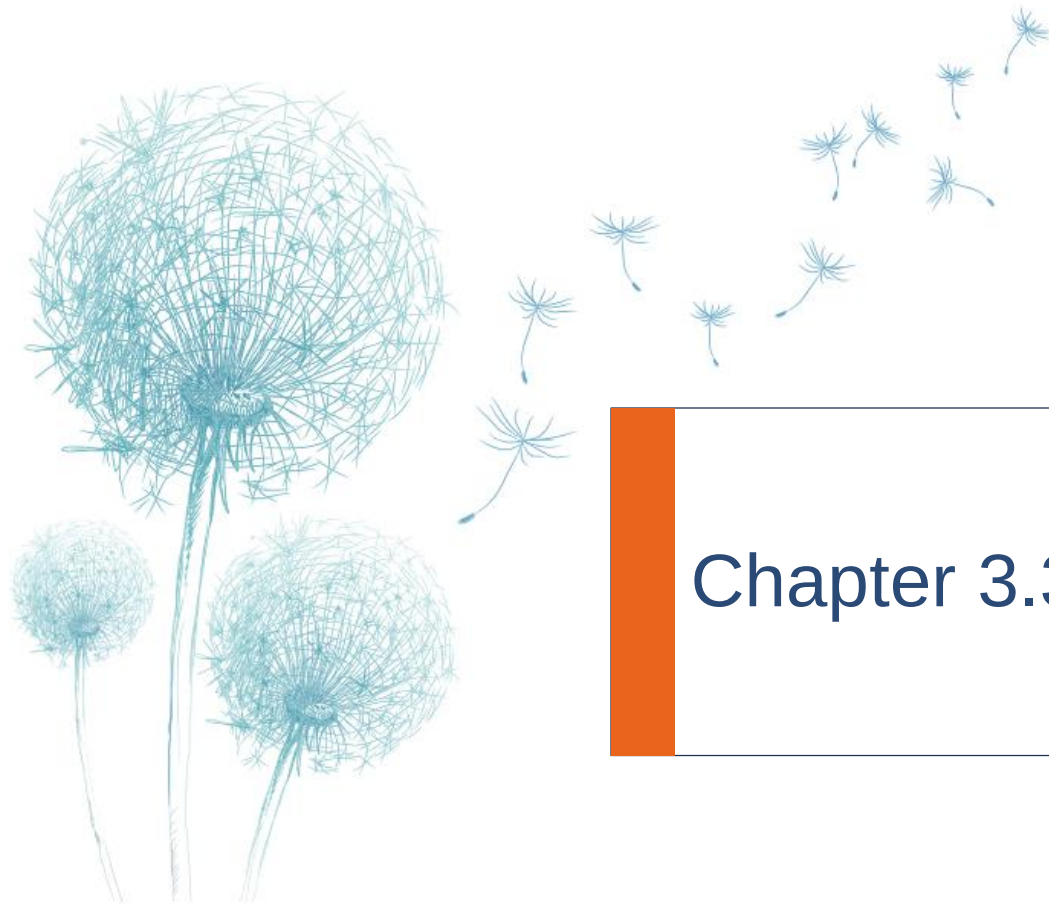
Repeat till convergence{

1. Select some pair α_i and α_j to update next
2. Reoptimize $W(\alpha)$ w.r.t α_i and α_j , while holding all the other α'_k s fixed

}

Support Vectors (Optional)

- ▶ After we find the α_i^* , based on the KKT condition, $\alpha_i^* g_i(w^*) = 0$, the data with $\alpha_i^* > 0$ are support vectors.
- ▶ For a solution $\alpha^* = (\alpha_1^*, \alpha_2^*, \dots, \alpha_m^*)^T$ for the dual problem, if exist α_j^* that $\alpha_j^* > 0$, then the solution for primal problem
 - ▶ $w^* = \sum_{i=1}^m \alpha_i^* y_i \mathbf{x}_i$
 - ▶ $b^* = y_j - \sum_{i=1}^m \alpha_i^* y_i \mathbf{x}_i^T \mathbf{x}_j$ (why?)



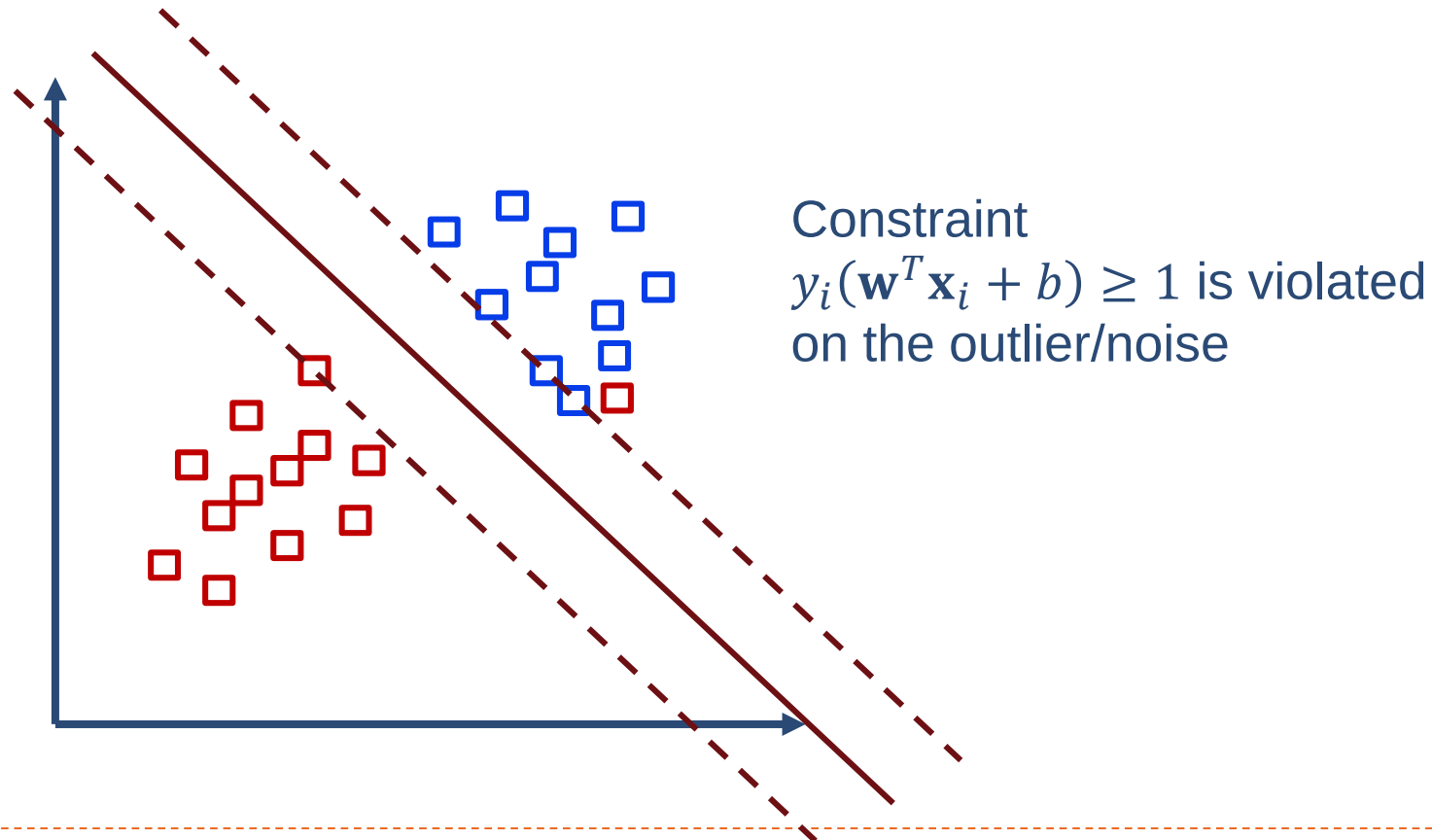
Chapter 3.3: Support Vector Machine II

Outline

- ▶ Support vector machine model
- ▶ Constrained optimization
- ▶ **Soft margin**
- ▶ **Kernel methods**

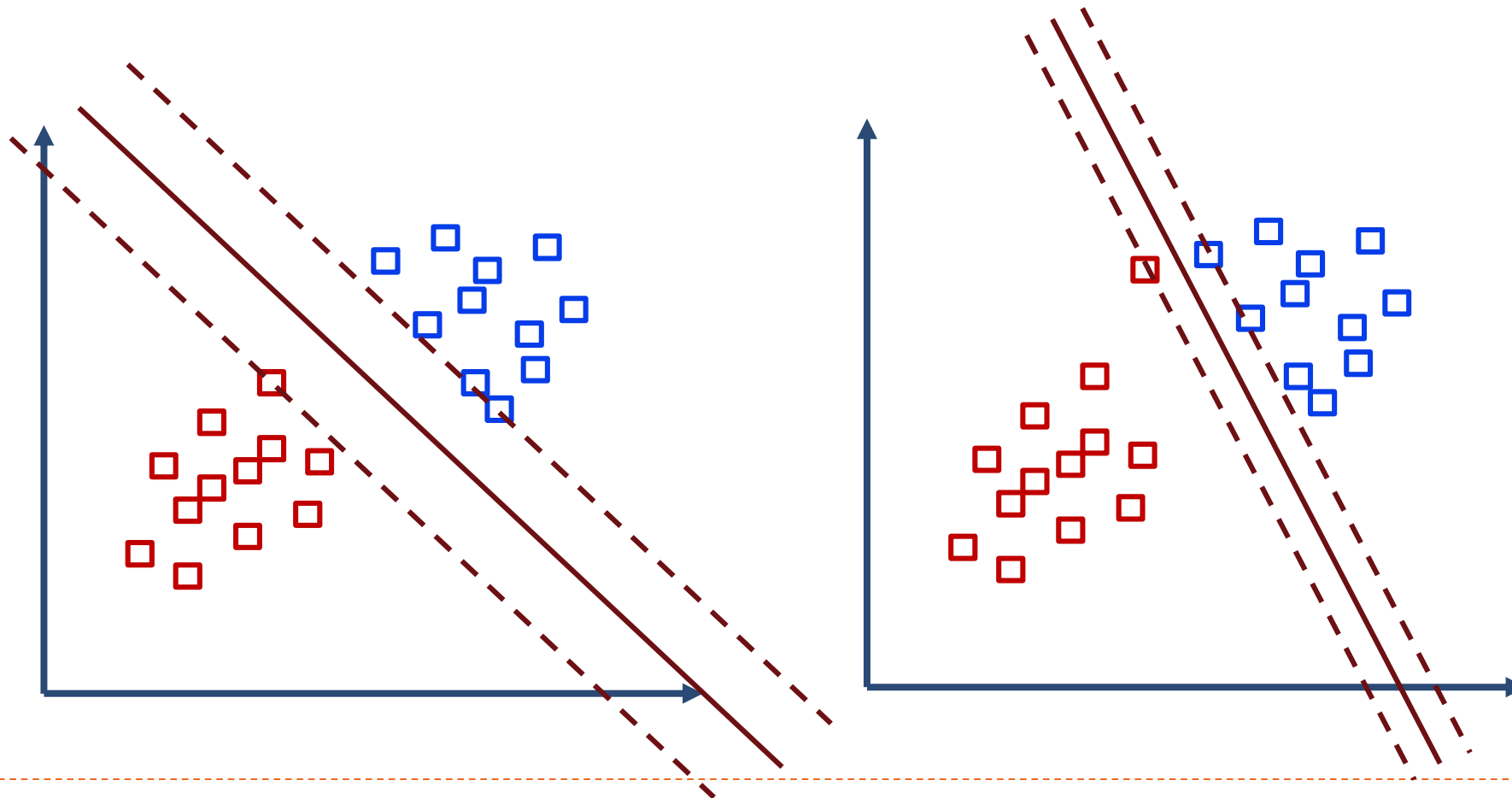
Non-linearly separable case

- What if training data not linearly separable due to noise or outliers?



Linearly separable with small margin

- Suppose we add an outlier which results in a classifier with a much smaller margin.



Soft Margin

- ▶ To make the SVM model work for non-linearly separable datasets as well as be less sensitive to outliers
 - ▶ allow error for some samples
- ▶ Allow some sample violated the constraint:
 - ▶ $y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 < 0$
 - ▶ soft margin, not hard margin
- ▶ Target: find a hyperplane that
 - ▶ maximize margin
 - ▶ minimize number of errors

Soft Margin

- ▶ Introduce positive slack variables ξ_i , constraints:

$$\begin{cases} \mathbf{w}^T \mathbf{x}_i + b \geq 1 - \xi_i & y_i = 1 \\ \mathbf{w}^T \mathbf{x}_i + b \leq -(1 - \xi_i) & y_i = -1 \\ \xi_i \geq 0 & \forall i \end{cases}$$

- ▶ New constraints:

- $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i$

- ▶ Penalize ξ_i in the objective function

- ▶ for an error to occur, the corresponding ξ_i must exceed unity

Optimization problem with soft margin

- ▶ The optimization problem using soft margin:

$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \xi_i \\ \text{s.t.} \quad & y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad i = 1, 2, \dots, m \\ & \xi_i \geq 0, \quad i = 1, 2, \dots, m \end{aligned}$$

- ▶ Examples are now permitted to have margin less than 1 and if an example whose margin is $1 - \xi_i$, we would pay a cost of the objective function being increased by $C \xi_i$
- ▶ Hinge loss: $l_{\text{hinge}}(z) = \max(0, 1 - z)$
- ▶ Another view of SVM:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \max(0, 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b))$$

Optimization problem with soft margin

- ▶ The parameter C controls the relative weighting between the twin goals of making
 - ▶ the $\|\mathbf{w}\|^2$ small
 - ▶ ensuring that most examples have margin greater than 1
- ▶ For large C value: the optimization will choose a **smaller-margin** hyperplane if that hyperplane does a better job of getting all the training points **classified correctly**.
- ▶ For small C value: the optimizer to look for a **larger-margin** separating hyperplane, even if that hyperplane **misclassifies more points**.

Optimization problem with soft margin (optional)

► Similarly, it is a constrained (convex) quadratic optimization problem. We construct the Lagrangian for this problem:

$$\begin{aligned} \text{► } \mathcal{L}(w, b, \xi, \alpha, \mu) = & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i + \\ & \sum_{i=1}^m \alpha_i [1 - \xi_i - y_i(\mathbf{w}^T \mathbf{x}_i + b)] - \sum_{i=1}^m \mu_i \xi_i \end{aligned}$$

► where $\alpha_i \geq 0, \mu_i \geq 0$ are lagrange multipliers

► Find the dual problem do by setting the derivatives of $\mathcal{L}$ with respect to w, b and ξ to zero:

$$\frac{\partial}{\partial w} \mathcal{L}(w, b, \xi, \alpha, \mu) = w - \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i = 0$$

$$\frac{\partial}{\partial b} \mathcal{L}(w, b, \xi, \alpha, \mu) = \sum_{i=1}^m -\alpha_i y_i = 0$$

$$\frac{\partial}{\partial \xi_i} \mathcal{L}(w, b, \xi, \alpha, \mu) = C - \alpha_i - \mu_i = 0$$

Optimization problem with soft margin (optional)

► Dual problem:

$$\begin{aligned} \max_{\alpha} W(\alpha) &= \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \\ \text{s.t. } C &\geq \alpha_i \geq 0, \quad i = 1, 2, \dots, m \\ \sum_{i=1}^m \alpha_i y_i &= 0 \end{aligned}$$

- There exists $w^*, b^*, \xi^*, \alpha^*, \mu^*$, so that w^*, b^*, ξ^* is the solution to the primal problem and α^*, μ^* are solution to the dual problem and $p^* = d^* = \mathcal{L}(w^*, b^*, \xi^*, \alpha^*, \mu^*)$
- Moreover, $w^*, b^*, \xi^*, \alpha^*, \mu^*$ satisfy the KKT conditions

Optimization problem with soft margin (optional)

- Similar with hard margin svm model, besides the partial derivative of w^*, b^*, ξ^* be zero, the KKT conditions are :

$$1 - \xi_i^* - y_i(\mathbf{w}^{*T} \mathbf{x}_i + b^*) \leq 0, \xi_i^* \geq 0, \forall i \quad (1)$$

$$\alpha_i^* \geq 0, \mu_i^* \geq 0, \forall i \quad (2)$$

$$\alpha_i^*(1 - \xi_i - y_i(\mathbf{w}^{*T} \mathbf{x}_i + b^*)) = 0, \forall i \quad (3)$$

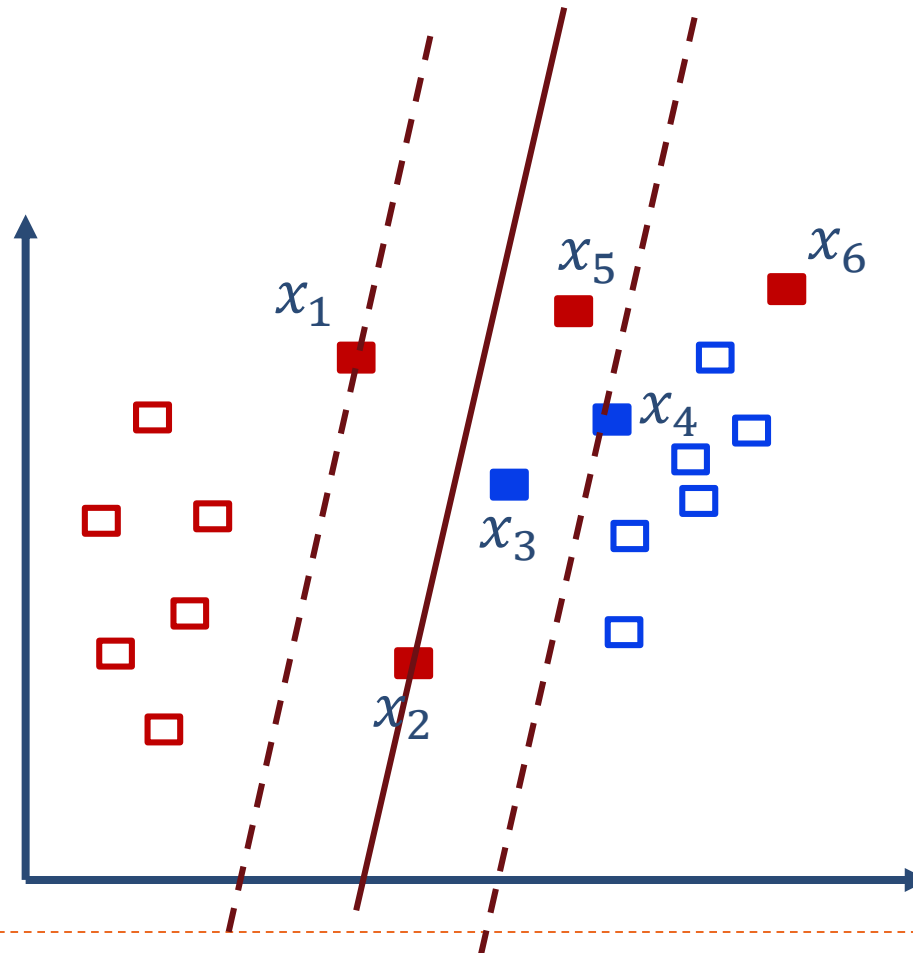
$$\mu_i^* \xi_i^* = 0, \forall i \quad (4)$$

Soft Margin SVM (optional)

- ▶ For any sample $(\mathbf{x}_i, y_i)$
 - ▶ $\alpha_i^* = 0, \mu_i = C$, therefore $\xi_i = 0$, sample lies beyond the margin.
 - ▶ $0 < \alpha_i^* < C, \mu_i > 0$, therefore $\xi_i = 0$ and $1 - \xi_i - y_i(\mathbf{w}^T \mathbf{x}_i + b) = 0$ sample lies on the maximum margin (support vectors)
 - ▶ $\alpha_i^* = C, \mu_i = 0$, if $\xi_i \leq 1$, sample lies inside the maximum margin; $\xi_i > 1$, misclassified (error)
- ▶ For a solution $\alpha^* = (\alpha_1^*, \alpha_2^*, \dots, \alpha_m^*)^T$ for the dual problem, if exist α_j^* that $0 < \alpha_j^* < C$, then the solution for primal problem
 - ▶ $\mathbf{w}^* = \sum_{i=1}^m \alpha_i^* y_i \mathbf{x}_i$
 - ▶ $b^* = y_j - \sum_{i=1}^m \alpha_i^* y_i \mathbf{x}_i^T \mathbf{x}_j$

Soft Margin SVM

- What are the values or range of ξ_i for the data samples in the given example?

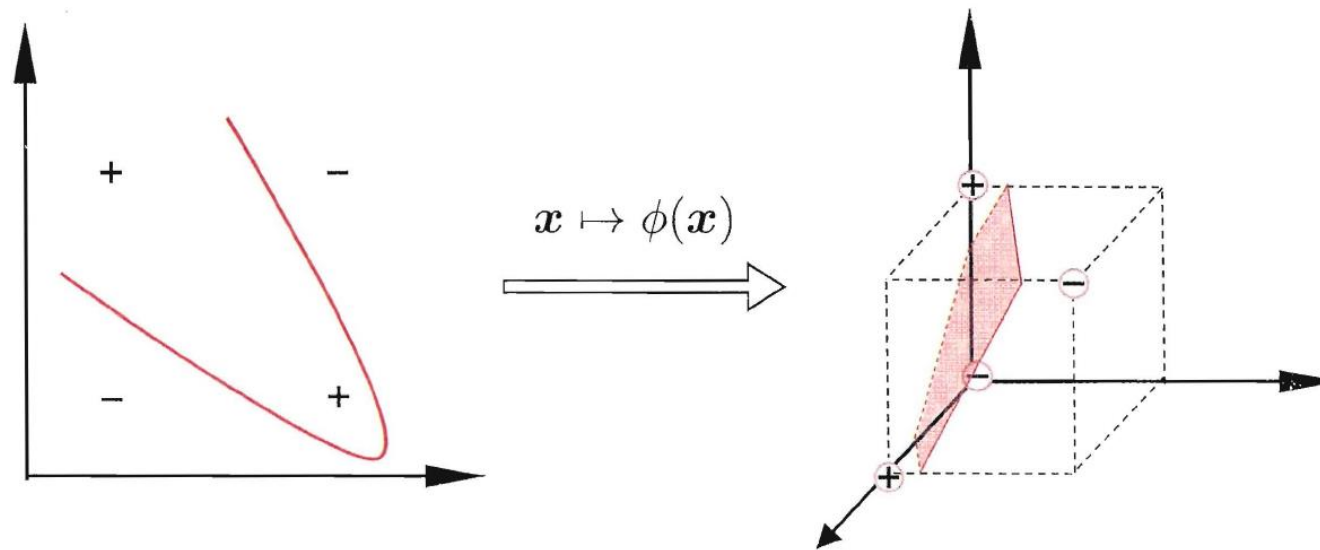


Nonlinear SVM

- ▶ What if training data not linearly separable?
 - ▶ Naïve: find non-linear decision surface
 - ▶ Not efficient
 - ▶ Feature transformation: map the original attributes into new feature space and apply **linear model** in the **new space**.

Feature Mapping

- ▶ Let φ denote the feature mapping
 - ▶ $\varphi: \mathbb{R}^m \rightarrow \mathcal{H}, \mathbf{x} \mapsto \varphi(\mathbf{x})$
 - ▶ where $\mathbb{R}^m$ is the input space and $\mathcal{H}$ is the mapped feature space



Feature Mapping

► Example:

- $\mathbf{x} = (x_1, x_2)^T \rightarrow \varphi(\mathbf{x}) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)^T$
- $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n) \rightarrow (\varphi(\mathbf{x}_1), y_1), \dots, (\varphi(\mathbf{x}_n), y_n)$
- $h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + b) \rightarrow \text{sign}(\mathbf{w}^T \varphi(\mathbf{x}) + b)$
- Apply Linear SVM on mapped feature space, the optimization problem is :

$$\begin{aligned} & \min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \\ & \text{s.t. } y_i(\mathbf{w}^T \varphi(\mathbf{x}_i) + b) \geq 1, \quad i=1, 2, \dots, m \end{aligned}$$

Feature Mapping

- ▶ The dual problem can be written entirely in terms of the inner products of input features $(\mathbf{x}_i^T \mathbf{x}_j)$, this means that we would replace all those inner products with $\varphi(\mathbf{x}_i)^T \varphi(\mathbf{x}_j)$.

- ▶ Dual problem:

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \varphi(\mathbf{x}_i)^T \varphi(\mathbf{x}_j) \\ \text{s.t.} \quad & \alpha_i \geq 0, \quad i = 1, 2, \dots, m \\ & \sum_{i=1}^m \alpha_i y_i = 0 \end{aligned}$$

- ▶ After solving the dual problem:

$$\begin{aligned} h(\mathbf{x}) &= \text{sign}(\mathbf{w}^T \varphi(\mathbf{x}) + b) \\ &= \text{sign}(\sum_{i=1}^m \alpha_i y_i \varphi(\mathbf{x}_i)^T \varphi(\mathbf{x}) + b) \end{aligned}$$

Feature Mapping

- ▶ Problem:

- ▶ $\varphi(\mathbf{x}_i)^T \varphi(\mathbf{x}_j)$: inner product of mapped feature vectors, may be expensive to calculate as $\varphi(\mathbf{x}_i)$ may be high dimensional vectors.
- ▶ For efficient calculation, define **kernel function**.

- ▶ Example:

- ▶ Given input instance with feature dimension 2 and the following feature mapping function
 - ▶ $\mathbf{x} = (x_1, x_2)^T \rightarrow \varphi(\mathbf{x}) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)^T$
 - ▶ $\varphi(\mathbf{x})^T \varphi(\mathbf{y}) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)(y_1^2, \sqrt{2}y_1y_2, y_2^2)^T$
 $= x_1^2y_1^2 + 2x_1x_2y_1y_2 + x_2^2y_2^2 = (\mathbf{x}^T \mathbf{y})^2$ (shortcut)
- ▶ Inner product can be computed in $\mathbb{R}^2$ (without going to $\mathcal{H}$).

Kernel Functions

- ▶ Kernel: Given a feature mapping φ , we define the corresponding Kernel to be:

$$\kappa(\mathbf{x}, \mathbf{y}) = \varphi(\mathbf{x})^T \varphi(\mathbf{y})$$

- ▶ we can get SVMs to learn in the high dimensional feature space given by φ , but without ever having to explicitly find or represent vectors $\varphi(x)$.
- ▶ Then, everywhere we have inner product of $\mathbf{x}, \mathbf{y}$ in our algorithm, we could replace it with $\kappa(\mathbf{x}, \mathbf{y})$. Our algorithm would be learning using the features φ .

Kernel Functions

- Dual problem can be re-write as:

$$\begin{aligned} \max_{\alpha} W(\alpha) &= \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \kappa(\mathbf{x}_i, \mathbf{x}_j) \\ \text{s.t. } \alpha_i &\geq 0, \quad i = 1, 2, \dots, m \\ \sum_{i=1}^m \alpha_i y_i &= 0 \end{aligned}$$

- Solving the dual problem and the classifier can be represented:

$$\begin{aligned} h(\mathbf{x}) &= \text{sign}(\mathbf{w}^T \varphi(\mathbf{x}) + b) \\ &= \text{sign}(\sum_{i=1}^m \alpha_i y_i \varphi(\mathbf{x}_i)^T \varphi(\mathbf{x}) + b) \\ &= \text{sign}(\sum_{i=1}^m \alpha_i y_i \kappa(\mathbf{x}_i, \mathbf{x}) + b) \end{aligned}$$

Kernel Functions - Examples

► Example:

► Consider Kernel function: $\kappa(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T \mathbf{y})^2$ ($\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$)

► $\kappa(\mathbf{x}, \mathbf{y}) = (\sum_{i=1}^n x_i y_i) (\sum_{i=1}^n x_i y_i) =$
 $\sum_{i=1}^n \sum_{j=1}^n x_i x_j y_i y_j = \sum_{i,j=1}^n (x_i x_j) (y_i y_j) = \varphi(\mathbf{x})^T \varphi(\mathbf{y})$

► $\varphi(\mathbf{x}) = \begin{bmatrix} x_1 x_1 \\ x_1 x_2 \\ \dots \\ x_n x_n \end{bmatrix}, \varphi(\mathbf{y}) = \begin{bmatrix} y_1 y_1 \\ y_1 y_2 \\ \dots \\ y_n y_n \end{bmatrix}$

► Calculate $\varphi(\mathbf{x})^T \varphi(\mathbf{y})$ requires $O(n^2)$, whereas calculate $\kappa(\mathbf{x}, \mathbf{y})$ requires $O(n)$.

Kernel Functions - Examples

► Consider another kernel function: $\kappa(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T \mathbf{y} + c)^2$ ($\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$)

$$\begin{aligned}\kappa(\mathbf{x}, \mathbf{y}) &= \left(\sum_{i=1}^n x_i y_i + c\right) \left(\sum_{i=1}^n x_i y_i + c\right) \\ &= \sum_{i=1}^n \sum_{j=1}^n x_i x_j y_i y_j + 2c \sum_{i=1}^n x_i y_i + c^2 \\ &= \sum_{i,j=1}^n (x_i x_j)(y_i y_j) + \sum_{i=1}^n (\sqrt{2c} x_i)(\sqrt{2c} y_i) + c^2\end{aligned}$$

► Corresponding to the feature mapping:

$$\varphi(\mathbf{x}) = (x_1 x_1, x_1 x_2, \dots, x_n x_n, \sqrt{2c} x_1, \sqrt{2c} x_2, \dots, \sqrt{2c} x_n, c)^T$$

Kernel Functions - Examples

- ▶ More broadly, the kernel $\kappa(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T \mathbf{y} + c)^d$ corresponds to a feature mapping to an $O(n^d)$ feature space (all terms are up to order d).
- ▶ However, despite working in this $O(n^d)$ -dimensional space, computing $\kappa(\mathbf{x}, \mathbf{y})$ still takes only $O(n)$ time, and hence we never need to explicitly represent feature vectors in this very high dimensional feature space.

Kernel Trick

- ▶ Now, let's talk about a slightly different view of kernels.
- ▶ Intuitively, if $\varphi(\mathbf{x})$ and $\varphi(\mathbf{y})$ are close together, then we might expect $\kappa(\mathbf{x}, \mathbf{y}) = \varphi(\mathbf{x})^T \varphi(\mathbf{y})$ to be large.
- ▶ Conversely, if $\varphi(\mathbf{x})$ and $\varphi(\mathbf{y})$ are far apart, say nearly orthogonal to each other, then $\kappa(\mathbf{x}, \mathbf{y}) = \varphi(\mathbf{x})^T \varphi(\mathbf{y})$ will be small.
- ▶ So, we can think of $\kappa(\mathbf{x}, \mathbf{y})$ as some measurement of how similar are $\varphi(\mathbf{x})$ and $\varphi(\mathbf{y})$, or of how similar are $\mathbf{x}$ and $\mathbf{y}$.

Kernel Trick

- ▶ Given this intuition, suppose that for some learning problem that you're working on, you've come up with some function $\kappa(\mathbf{x}, \mathbf{y})$ that you think might be a reasonable measure of how similar $\mathbf{x}$ and $\mathbf{y}$ are.

- ▶ For instance, choose:

$$\kappa(\mathbf{x}, \mathbf{y}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{y}\|^2}{2\sigma^2}\right)$$

- ▶ This is a reasonable measure of $\mathbf{x}$ and $\mathbf{y}$'s similarity, and is close to 1 when $\mathbf{x}$ and $\mathbf{y}$ are close, and near 0 when they are far apart.
- ▶ Gaussian kernel, corresponds to an infinite dimensional feature mapping φ .

Kernel Trick

- ▶ But more broadly, given some function κ , how can we tell if it's a valid kernel
 - ▶ Can we tell if there is some feature mapping φ so that $\kappa(\mathbf{x}, \mathbf{y}) = \varphi(\mathbf{x})^T \varphi(\mathbf{y})$ for all $\mathbf{x}, \mathbf{y}$?
- ▶ Theorem (Mercer):
 - ▶ Let $\kappa: \mathbb{R}^n \times \mathbb{R}^n \mapsto \mathbb{R}$ be given. Then for κ to be a valid kernel, it is necessary and sufficient that for finite number of points $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$, the corresponding kernel matrix is symmetric positive semi-definite.
 - ▶ Kernel matrix for m points are $m \times m$ matrix K , and the (i, j) -entry is $K_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j)$.
- ▶ In real application, usually use the existing kernel functions.

Kernel Functions

- ▶ Common kernels:

- ▶ Linear: $\kappa(\mathbf{x}, \mathbf{y}) = \mathbf{x}^T \mathbf{y}$

- ▶ Polynomial: $\kappa(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T \mathbf{y} + c)^d$

- ▶ d is the degree of the polynomial

- ▶ Gaussian: $\kappa(\mathbf{x}, \mathbf{y}) = \exp(-\frac{\|\mathbf{x}-\mathbf{y}\|^2}{2\sigma^2})$ (or $\kappa(\mathbf{x}, \mathbf{y}) = \exp(-\gamma\|\mathbf{x} - \mathbf{y}\|^2)$)

- ▶ Corresponds to an infinite dimensional feature space

- ▶ Usually the first choice.

- ▶ radial basis function (RBF)

- ▶ https://scikit-learn.org/stable/auto_examples/svm/plot_rbf_parameters.html

Kernel Functions

- ▶ Kernel: substitutes the inner product and act as a nonlinear similarity measure
- ▶ Kernel implicitly defines the feature space
- ▶ Any algorithm that depends only on inner products can use the kernel trick!
- ▶ Perform feature scaling before applying kernel trick.

How to Choose Kernel Function

- ▶ Select kernel function based on expert or prior knowledge of the data; if no prior knowledge, try cross validation.
 - ▶ Grid search for hyper-parameter/parameter setting.
 - ▶ <https://www.csie.ntu.edu.tw/~cjlin/papers/guide/guide.pdf>
- ▶ Suppose n is the number of features, m is the number of samples
 - ▶ If n is large (compare to m): linear kernel.
 - ▶ If n is small, m is intermediate: RBF kernel
 - ▶ If n is small, m is large: create/add more features and use linear model.